

Chapter 7 — Vectors: Dynamically Resizable Arrays in C++

The STL `std::vector`: size vs capacity, reallocation, insertion/erasure, and multidimensional vectors

How to read these notes

PROJECTS Section 7.5 is provided mainly to help on projects (memory optimization of multidimensional vectors). **Everything else in this chapter is standard class material you are responsible for knowing.**
Blue boxes = remarks / function specifications. Green boxes = worked examples. Orange boxes = pitfalls.

7.1 The STL Vector Container

- You do not have to implement a dynamic array-based container from scratch — the **C++ standard template library (STL)** provides efficient containers and algorithms.
- `std::vector` = an array-based container that manages its elements using a **dynamically-sized, heap-allocated C-style array**.
 - Stores elements much like an array, but **manages its own memory** and **resizes itself automatically** when more space is needed (an array is fixed in size).
 - Requires `#include <vector>`.
- Example: `std::vector<int32_t> vec = {1, 2, 3, 4};` creates a C-style array on the **heap** with contents {1, 2, 3, 4}.

Under the hood

- Implementation-specific, but the example implementation used here (based on the GCC libstdc++ version) stores **three pointers on the program stack**:
 - `begin` — points to the **beginning** of the heap-allocated array.
 - `end_size` — points **one past the last valid element**.
 - `end_cap` — points **one past the last allocated element**.
- The last valid element and the last allocated element **need not be the same**.

begin	end_size	end_cap
0x100	0x110	0x110

heap:	1	2	3	4
	0x100	0x104	0x108	0x10c

Note: this is only one possible implementation

Exact details may be platform dependent, as long as the public interface conforms to the C++ standard. Another valid implementation could keep **one pointer plus two integers** (size and capacity) instead of three pointers. The point is to understand the general idea, **not to memorize this implementation**.

Size vs capacity

- Regardless of implementation, a vector must always be able to determine two values:
 - size** = number of **valid elements** the underlying array actually holds.
 - capacity** = number of elements the underlying array is **able to hold**.
 - These two values **do not need to be the same**.
- Computed from the pointers: **size** = `end_size - begin` and **capacity** = `end_cap - begin`.
- Example: remove the last element (4) from the vector above. The underlying array still holds at most 4 elements, but only 3 are valid → **size 3, capacity 4**.

begin	end_size	end_cap
0x100	0x10c	0x110

1	2	3	undef
0x100	0x104	0x108	0x10c

Footnote: pointer arithmetic

Pointer arithmetic is performed relative to the **base type** of the pointer. Since integers take 4 bytes each, incrementing 0x100 by 1 actually gives $0x100 + 1 \times 4 = 0x104$.

Vector constructors

Constructor	Behavior & example
<code>std::vector<T>();</code>	Default constructor. Creates an empty vector with no elements; size and capacity initially 0. <code>std::vector<int32_t> v1;</code>
<code>std::vector<T>(std::initializer_list<T> init);</code>	Initializes the vector with the contents of the initializer list. <code>std::vector<int32_t> v2{1, 2, 3};</code> <code>std::vector<int32_t> v3 = {1, 2, 3};</code> → contents {1,2,3}, size and capacity 3
<code>std::vector<T>(const std::vector<T>& other);</code>	Copy constructor , copies the contents of <code>other</code> . <code>std::vector<int32_t> v4{v3};</code> → {1, 2, 3}
<code>std::vector<T>(size_t sz);</code>	Creates a vector of <code>sz</code> elements, each value initialized ; size and capacity both = <code>sz</code> . <code>std::vector<int32_t> v5(5);</code> → size and capacity 5, each element 0
<code>std::vector<T>(size_t sz, T& val);</code> (the "fill constructor")	Creates a vector of <code>sz</code> elements, each initialized to <code>val</code> ; size and capacity both = <code>sz</code> . <code>std::vector<int32_t> v6(5, 1);</code> → size and capacity 5, each element 1
<code>std::vector<T>(InputIterator begin_iter, InputIterator end_iter);</code>	Creates a vector with all elements in the iterator range [begin_iter , end_iter) — inclusive begin, exclusive end. <code>std::vector<int32_t> v7{v6.begin(), v6.begin() + 3};</code> → first three elements of <code>v6</code> <code>.begin()</code> returns an iterator to the first element of the vector.

7.2 Inserting and Removing Elements

7.2.1 Push Back

```
template <typename T>
void std::vector<T>::push_back(const T& val);
```

- Appends a new value initialized to `val` to the **back** of the vector, after the current last element (if there is available capacity).
- If the new size would surpass the current capacity, the underlying array is **reallocated first**, then the element is appended.
- After a successful append, **size increases by one**.

Reallocation walkthrough — vector with size 4, capacity 4, then `vec.push_back(5)`

- **Step 1.** Allocate a new array with **double** the capacity (8) elsewhere in memory.
- **Step 2.** Copy the contents from the old array to the new array.
- **Step 3.** Deallocate (free) the old array and **reset the internal pointers** to the new array.
- **Step 4.** Push back the new value.

begin	end	size	end_cap				
0x200	0x214	0x220					
1	2	3	4	5	undef	undef	undef
0x200	0x204	0x208	0x20c	0x210	0x214	0x218	0x21c

7.2.2 Pop Back

```
template <typename T>
void std::vector<T>::pop_back();
```

- Removes the last element in the vector, **reducing its size by one**.
- **Does not trigger a reallocation.** After popping the 5 from the vector above: size becomes 4, but **capacity is still 8**.
- Reallocation can only happen if (a) an insertion occurs when the vector is at capacity, or (b) the programmer explicitly shrinks or expands the capacity.
- Calling `.pop_back()` on an **empty vector is undefined behavior**. The programmer is expected to know whether the vector is empty; this removes the need for an extra (potentially unnecessary) "empty" check on every call — **sacrificing safety for performance**.

Complexity and amortization

- `.pop_back()` takes $\Theta(1)$.
- `.push_back()` takes $\Theta(n)$ if reallocation occurs, $\Theta(1)$ otherwise.
- Because capacity is **doubled** on reallocation, each $\Theta(n)$ push makes subsequent pushes cheaper. Averaging the reallocation work across all individual pushes lets `.push_back()` be treated as a $\Theta(1)$ operation. This idea is called **amortization** (chapter 12).

7.2.3 Emplace Back

```
template <typename T, typename... Args>
T& std::vector<T>::emplace_back(Args&&... args);
```

- Appends a new element at the back, right after the current last element (if capacity allows); reallocates first if the new size surpasses capacity.
- The element is **constructed in place** using the arguments for its constructor (`args`), which are passed into the `.emplace_back()` call.
- After a successful `emplace`, size increases by one. Since C++17, a **reference to the emplaced element is returned**.
- Useful alternative to `.push_back()` for vectors of **large objects**.

```
struct Student {
    int32_t id;
    std::string name;
    double gpa;
    Student(int32_t id_in, std::string name_in, double gpa_in) :
        id{id_in}, name{name_in}, gpa{gpa_in} {}
};

std::vector<Student> students;
```

push_back — temporary + copy

```
Student s{12345678, "Bob", 3.5};
students.push_back(s);
// or in one line:
students.push_back(Student{12345678, "Bob", 3.5});
```

You must first create a **temporary Student** (`.push_back()` only accepts an argument of the vector's type). The temporary is then **copied** to the back of the vector.

emplace_back — constructed in place

```
// pass what you'd pass to the constructor
students.emplace_back(12345678, "Bob", 3.5);
```

Accepts a **list of constructor arguments** instead of a `Student`. It **forwards** them to a constructor that builds the object directly at the destination memory address inside the vector. **No temporary copy is needed.**

7.2.4 Inserting and Erasing From Any Position

- Vectors require elements to be stored **contiguously** in memory:
 - **Insert:** shift all elements after the insertion point **rightward** to make room.
 - **Erase:** shift all elements after the deletion point **leftward** so there are no gaps.
- `.push_back()`, `.emplace_back()`, and `.pop_back()` all act at the **end**, so nothing needs to be shifted → all $\Theta(1)$, assuming no reallocation.

1 3 4 5 → 1 3 4 5 → 1 2 3 4 5

Insert

```
iterator std::vector<T>::insert(const_iterator position, const T& val);
```

- Inserts `val` **directly before** the element pointed to by `position`; returns an iterator to the newly inserted element. Reallocates if the new size exceeds capacity.

```
std::vector<int32_t> vec = {0, 1, 2, 4, 5};
vec.insert(vec.begin() + 3, 3);    // insert 3 at index 3
// vec now {0, 1, 2, 3, 4, 5}
```

```
iterator std::vector<T>::insert(const_iterator position, size_t n, const T& val);
```

- Inserts **n copies** of **val** directly before **position**; returns an iterator to the **first** new element added.

```
std::vector<int32_t> vec = {0, 1, 2, 4, 5};
vec.insert(vec.begin() + 3, 4, 3);    // insert 4 copies of 3 at index 3
// vec now {0, 1, 2, 3, 3, 3, 3, 4, 5}
```

```
iterator std::vector<T>::insert(const_iterator position, InputIterator first, InputIterator last);
```

- Inserts all elements in the iterator range [**first**, **last**) directly before **position**; returns an iterator to the first new element added.

```
std::vector<int32_t> vec1 = {0, 1, 5};
std::vector<int32_t> vec2 = {2, 3, 4};
vec1.insert(vec1.begin() + 2, vec2.begin(), vec2.end());
// vec1 now {0, 1, 2, 3, 4, 5}
```

```
iterator std::vector<T>::insert(const_iterator position, std::initializer_list<T> init);
```

- Inserts the elements of the initializer list directly before **position**; returns an iterator to the first new element added.

```
std::vector<int32_t> vec = {0, 1, 2};
vec.insert(vec.end(), {3, 4, 5});    // insert {3, 4, 5} at the back
// vec now {0, 1, 2, 3, 4, 5}
```

- **Complexity of .insert()**: linear on **(the number of elements inserted) + (the number of elements after the insertion point)**, since each of those must be shifted over.

Emplace (construct in the middle)

```
template <typename T, typename... Args>
iterator std::vector<T>::emplace(const_iterator pos, Args&&... args);
```

- Inserts a new element directly before the element at **pos**, **constructed in place** from the constructor arguments. Returns an iterator to the emplaced object.

```
std::vector<Student> students;
students.emplace_back(12345677, "Alice", 3.4);
students.emplace_back(12345679, "Cathy", 3.6);
students.emplace(students.begin() + 1, 12345678, "Bob", 3.5);
// students now in this order: {Alice, Bob, Cathy}
```

Erase

```
iterator std::vector<T>::erase(const_iterator position);
iterator std::vector<T>::erase(const_iterator first, const_iterator last);
```

- First form: erases the element pointed to by **position**. Second form: erases all elements in the range [**first**, **last**).
- Both return an iterator pointing to the **new location of the element that followed** the erased element(s).

```
std::vector<int32_t> vec = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
vec.erase(vec.begin() + 6, vec.end());    // erase index 6 to the end
// vec now {0, 1, 2, 3, 4, 5}
vec.erase(vec.begin() + 3);                // erase element at index 3
// vec now {0, 1, 2, 4, 5}
```

- **Complexity of .erase()**: linear on **(the number of elements erased) + (the number of elements after the erasure point)**, since elements must be shifted so there are no gaps.
- Both **.insert()** and **.erase()** use **iterators**: generalized pointers used to traverse a container's contents (chapter 11).

Iterator-returning member functions

Function	Behavior
.begin()	Random access iterator to the first element in the vector
.end()	Random access iterator to the position one past the last element
.cbegin() / .cend()	Constant random access iterator to the first element / one past the last element
.rbegin() / .rend()	Reverse iterator to the last element / one before the first element
.crbegin() / .crend()	Constant reverse iterator to the last element / one before the first element

Other supported operations

Function	Behavior
operator[]	Returns a reference to the element at a given index
.front()	Returns a reference to the first element (undefined behavior if empty)
.back()	Returns a reference to the last element (undefined behavior if empty)
.empty()	Returns whether the vector is empty (true or false)
.clear()	Erases all elements, leaving size 0 (but capacity unchanged)

```
std::vector<int32_t> vec = {183, 203, 280, 281};
std::cout << vec.front() << '\n'; // prints 183
std::cout << vec.back() << '\n'; // prints 281
std::cout << vec[1] << '\n'; // prints 203
vec.clear();
std::cout << vec.empty() << '\n'; // prints 1 (true)
```

7.3 Resize and Reserve

7.3.1 The Costs of Reallocation

Function	Behavior
<code>.size()</code>	Returns the size of the vector
<code>.capacity()</code>	Returns the capacity of the vector's underlying data array

Growth pattern

- An empty vector starts with size 0 and capacity 0. Pushing the first element allocates an array and capacity becomes **1**.
- From then on, most implementations **double** capacity on each reallocation: 1 → 2 (2nd element) → 4 (3rd) → 8 (5th) → 16 (9th) → ...
This is the behavior if every element is added with `.push_back()`.
- If instead you initialize the vector with a non-zero number of elements (range constructor or initializer list), the initial size and capacity are typically **the number of elements present at initialization**.
 - `std::vector<int32_t> vec = {1, 2, 3, 4, 5};` → size 5, capacity 5.
 - One more `.push_back()` → size 6, capacity **doubles to 10**.
- Exact behavior may depend on the standard library implementation, but the **distinction between size and capacity** and the associated tradeoffs always hold.

Three problems caused by reallocation

- 1. Reallocation invalidates all existing pointers and iterators to elements in the vector.**
 - When data exceeds capacity, the vector moves the data to a larger array **elsewhere in memory**, so any address a pointer/iterator held in the old array is no longer valid.

```
std::vector<int32_t> vec = {1, 2, 3, 4}; // size 4, capacity 4
int32_t* ptr = &vec[3]; // ptr == 0x10c (where the 4 lives)
vec.push_back(5); // triggers reallocation to 0x200...
// the 4 has MOVED; it is no longer at 0x10c -> ptr is now invalid
```

- 2. Reallocation takes time.** All elements must be shifted from the old array to the new one. Pushing back 1,000,000 elements into a vector that doubles on each reallocation makes the reallocation process happen **21 times**.
- 3. Letting the vector set the capacity for you can waste memory** if you know your intended size.
 - Storing 1,025 elements with only `.push_back()`: capacity doubles 1 → 2 → 4 → 8 → 16 → 32 → 64 → 128 → 256 → 512 → 1,024. The 1,025th element forces a double to **2,048**.
 - The vector doesn't know element 1,025 was the last one. You reserve memory for 2,048 elements while needing 1,025 — the rest is unused and wasted.

Footnote: pointer/iterator invalidation

Invalidation is a pesky bug that is hard to detect. Be careful with pointers or iterators if you modify the contents of the underlying container after creating them. If you need guaranteed stability of references and iterators, containers like the boost library's `boost::container::stable_vector<>` keep references and iterators valid as long as the element is not erased. Tools like **valgrind** (chapter 6) can also help debug invalidation issues.

7.3.2 Reserving a Vector's Capacity

```
template <typename T>
void std::vector<T>::reserve(size_t n);
```

- Requests that the vector's capacity be **at least n**.
 - If `n > current capacity` → the vector **reallocates** its data to an array with capacity `n`.
 - Otherwise, no reallocation occurs.
- Does not affect the size of the vector.** You are not creating elements — only allocating space for future ones.

```
std::vector<int32_t> vec;
vec.reserve(10); // size still 0; capacity 10
```

begin	end_size	end_cap
0x270	0x270	0x298

undef	undef	undef	undef	undef	undef	undef	undef	undef	undef
0x270	0x274	0x278	0x27c	0x280	0x284	0x288	0x28c	0x290	0x294

- A `.push_back()` would place the new element at the **first open location** (0x270).
- Reserving above the current capacity causes reallocation: a vector `{1,2,3,4}` (capacity 4) after `vec.reserve(7)` has **size still 4, capacity 7**, and the next `.push_back()` goes to the first open position.

7.3.3 Resizing a Vector's Size

```
template <typename T>
void std::vector<T>::resize(size_t n);
void std::vector<T>::resize(size_t n, T& val);
```

- Resizes the vector so it contains **n elements**:
 - `n < current size` → the vector is forced down to size `n`; extra elements are **destroyed** (capacity unchanged).
 - `n > current size` → new elements are **value-initialized** and added to the end until size is `n`.

- $n >$ current capacity → **reallocation occurs** and the capacity also becomes n .
- Second overload: same, but new elements are **copies of `val`** instead of value-initialized.

Footnote: the capacity rule is an oversimplification

For the GCC `std::vector<>` implementation at the time of writing, if n is larger than the current size, capacity only becomes n if $n > 2 \times (\text{previous size})$. If $n < 2 \times (\text{previous size})$, capacity is set to $2 \times (\text{previous size})$ instead of n . Don't memorize these rules — they are implementation-specific and not stated in the standard.

Example: `resize` actually creates elements

```
std::vector<int32_t> vec = {1, 2, 3, 4}; // size 4, capacity 4
vec.resize(9);                          // size 9, capacity 9
```

1	2	3	4	0	0	0	0	0
---	---	---	---	---	---	---	---	---

- The last five elements are **value-initialized to 0** (default for integers); both size and capacity become 9.
- `vec.push_back(5)` now does **not** overwrite an existing slot — it adds after the 9th element. Since capacity is 9, a reallocation occurs and **capacity doubles to 18**.

`reserve` + `push_back` vs `resize` + `operator[]`

- Both save memory (allocate only what you intend to use) and speed up your program (fewer reallocations) if you know the size in advance. But **the way you add values differs**:

Reserve → use <code>.push_back()</code>	Resize → use <code>operator[]</code>
<pre>std::vector<int32_t> v1; v1.reserve(5); // reserve capacity to 5 v1.push_back(1); // add elements to back v1.push_back(2); // using .push_back() v1.push_back(3); v1.push_back(4); v1.push_back(5);</pre>	<pre>std::vector<int32_t> v2; v2.resize(5); // resize size to 5 v2[0] = 1; // add elements by v2[1] = 2; // modifying the values v2[2] = 3; // that were created v2[3] = 4; v2[4] = 5;</pre>
<code>.reserve()</code> creates no elements , so elements must be appended.	<code>.resize()</code> creates elements, so <code>.push_back()</code> would add after them (unintended behavior, and may even trigger a reallocation).

Remark: the refrigerator analogy

- You must carry a single portable refrigerator of water bottles to a sporting event, and must have enough bottles for demand.
- **Refrigerator** = the underlying fixed-size heap array. **Water bottles** = the stored data values. **You** = the vector, responsible for managing both. Only one refrigerator = a vector manages only one data array.
- `.resize(50)` = change the number of valid elements to 50 → add or remove water bottles until exactly 50 remain.
- `.reserve(50)` = change the capacity of the underlying array → buy a fridge that fits 50 bottles. **It does not change size**, just as buying a bigger fridge does not change how many bottles you have.
- More bottles than fridge space → buy a larger fridge (allocate a new, larger array), move all bottles over (copy elements), and discard the old fridge (deallocate the old array).

Example 7.1

```
1 std::vector<int32_t> vec;
2 vec.reserve(10);
3 vec[0] = 5;
4 bool b = vec.empty();
```

Line 3 is undefined behavior. `.reserve()` changes only the capacity of the underlying array, not the contents of the vector. The vector is still empty, so indexing accesses **uninitialized memory** (there is no element at index 0 yet).

Line 4 assigns `b` to `true`, since the vector is indeed empty (its size is 0).

Example 7.2

```
1 std::vector<int32_t> vec;
2 vec.resize(10);
3 vec.push_back(1);
4 std::cout << vec.size() << '\n';
```

`.resize()` changes the **size**, so line 2 creates 10 elements. Line 3 pushes back an element **after** those 10, making the size 11. **Line 4 prints 11.**

7.3.4 Accessing a Vector's Underlying Data Array

STARRED IN THE TEXT Marked as supplementary in the notes; included for completeness.

- `operator[]` retrieves an element **by reference** (see section 6.7 for the reasoning).
- To get a pointer to the **underlying array**: calling `&vec` does **not** work — it gives a pointer to the `std::vector` object **on the stack** (the three pointers), not the data on the heap.

```
template <typename T>
T* std::vector<T>::data();
```

- Returns a pointer to the vector's **underlying data array**. `vec.data()` is what you want.
- **Remark:** `&vec[0]` also returns a pointer to the underlying data array, but it is **undefined behavior if the vector is empty**. `.data()` is preferred — it is always safe regardless of whether the vector is empty.

7.4 Storing Multidimensional Data in Vectors

- 2-D data = a **vector of vectors**. The outer vector represents one dimension (e.g. rows), the inner vectors another (e.g. columns).
`std::vector<std::vector<int32_t>> vec_2d;`
- 3-D data = a vector storing 2-D vectors (each layer is a dimension):
`std::vector<std::vector<std::vector<int32_t>>> vec_3d;`
- Access with `operator[]` multiple times: `vec[2][3]` = row index 2, column index 3. This works because `vec[2]` is itself a vector object and can be indexed just like `vec`.

Remark: "2-D vector" is a shorthand

A multidimensional vector is **merely a one-dimensional vector that stores other vectors**. The "2-D vector" above is actually a one-dimensional vector of `std::vector<int32_t>` behind the scenes, not a two-dimensional vector of `int32_t`.

Filling a multidimensional vector

Naive approach — push_back everything (wasteful)

```
std::vector<std::vector<int32_t>> vec_2d;
while (vec_2d.size() < 5) {
    std::vector<int32_t> temp;
    while (temp.size() < 5) {
        temp.push_back(1);
    } // while
    vec_2d.push_back(temp);
} // while
```

- **Problem:** the size was never specified, so every vector **doubled with each reallocation** and each dimension ends up with a **capacity of 8 instead of 5** — three wasted slots at every level (outer vector and all five inner vectors).

Better — reserve the outer vector and fill-construct the inner ones

```
vector<vector<int32_t>> vec_2d;
vec_2d.reserve(5);
while (vec_2d.size() < 5) {
    vector<int32_t> temp(5, 1); // 5 elements, each initialized to 1
    vec_2d.push_back(temp);
} // while
```

Best — the fill constructor, on one line

- Recall: `std::vector<T> vec(size_t sz, T& val);` creates a vector of size `sz` with each element initialized to `val`.
- The fill constructor also **sets the capacity equal to the specified size**, so the underlying array has **exactly the right capacity** (5 instead of 8).
- For a 2-D vector, set `val` to the constructor of the inner vector:

```
std::vector<std::vector<T>> vec_2d(m, std::vector<T>(n, data));
```

- `m` = size of the outer vector, `n` = size of each inner vector, `data` = value each element is initialized to.
- A 5×5 vector of 1's — identical to the whole while loop above, but clean, efficient, and with no wasted capacity:

```
std::vector<std::vector<int32_t>> vec_2d(5, std::vector<int32_t>(5, 1));
//           ^outer size   ^inner vector: size 5, all 1's
```

- More dimensions = more nesting. A $2 \times 5 \times 10$ vector of 1's:

```
std::vector<std::vector<std::vector<int32_t>>> vec_3d(
    2, std::vector<std::vector<int32_t>>(5, std::vector<int32_t>(10, 1)));
```

7.5 Vector Performance and Memory Overhead PROJECTS

Per course directions: this section is provided mainly to help you on projects (optimizing multidimensional vector dimensions to reduce memory usage). All other chapter 7 material is standard class content.

Order of dimensions affects speed

- Common convention: the **row** dimension is the outer vector, the **column** dimension is the inner vectors (each index of the outer vector stores a row of data).
- Sequential access of memory addresses that are **close together** is faster than accesses that are far apart. So the dimension you plan to iterate through **first** should be the **outermost** vector, if possible.
 - Loop rows, then columns → **faster** (contiguous accesses), assuming rows are the outer dimension.
 - Loop columns, then rows → **slower** (non-contiguous accesses).
- Different problems need different iteration orders. If a problem requires visiting a 2-D vector **column-wise**, store **columns as the outer vector** and rows as the inner vectors to take advantage of fast sequential access.
 - If you do this you must **flip the indices**: `vec_2d[row][col]` becomes `vec_2d[col][row]`.
 - Tradeoff: storing columns as the outer dimension speeds up column-wise access but **slows down row-wise access**.

Order of dimensions also affects memory usage

- Counterintuitive, but true. For $2 \times 5 \times 10$ data you always store 100 values (400 bytes for `int32_t`, assuming proper `resize/reserve`) — but **each vector object also stores 3 bookkeeping pointers** (`begin`, `end_size`, `end_cap`).
- Each pointer takes **8 bytes** on a 64-bit machine, so differences accumulate quickly.

Case A: $2 \times 5 \times 10$

```
std::vector<std::vector<std::vector<int32_t>>> vec_3d(
    2, std::vector<std::vector<int32_t>>(5, std::vector<int32_t>(10, 1)));
```

Level	Pointers
1 outermost vector	3
2 vectors of size 5 (outermost dimension is 2)	$2 \times 3 = 6$
$2 \times 5 = 10$ vectors of size 10	$2 \times 5 \times 3 = 30$
Data level (stores integers, not vectors)	0
Total	$3 + 6 + 30 = 39$ pointers

Case B: dimensions flipped to $10 \times 5 \times 2$

```
std::vector<std::vector<std::vector<int32_t>>> vec_3d(
    10, std::vector<std::vector<int32_t>>(5, std::vector<int32_t>(2, 1)));
```

Level	Pointers
1 outermost vector	3
10 vectors of size 5	$10 \times 3 = 30$
$10 \times 5 = 50$ vectors of size 2	$10 \times 5 \times 3 = 150$
Data level	0
Total	$3 + 30 + 150 = 183$ pointers — over 4× as many

General rule

For a 3-D vector with dimensions a (outermost), b , c (innermost):

```
std::vector<std::vector<std::vector<T>>> vec_3d(a, std::vector<std::vector<T>>(b, std::vector<T>(c, VAL)));
```

the number of bookkeeping pointers is **$3 + 3a + 3ab$** .

Therefore, order the dimensions **from smallest to largest** ($a < b < c$) to minimize memory overhead. This also holds for more than 3 dimensions (provable by the same process) — e.g. a 4-D vector with dimensions a, b, c, d needs **$3 + 3a + 3ab + 3abc$** pointers, so the **smallest** dimension should be the outermost.

7.6 Summary of Vector Complexities

Operation	Average	Worst	Why / STL function
Finding an element	$\Theta(n)$	$\Theta(n)$	Worst case: the element is last or not present, so all n elements are examined. Average: element is in the middle, so about $n/2$ elements are examined — still $\Theta(n)$ after dropping the $1/2$ coefficient. Uses <code>std::find()</code> in <code><algorithm></code> (chapter 11).
Accessing first element	$\Theta(1)$	$\Theta(1)$	The vector keeps a pointer to the first element of its array. <code>std::vector::front()</code>
Accessing last element	$\Theta(1)$	$\Theta(1)$	The vector keeps a pointer one past the last element. <code>std::vector::back()</code>
Accessing arbitrary element	$\Theta(1)$	$\Theta(1)$	Elements are contiguous, so pointer arithmetic reaches any index in constant time. <code>operator[]</code> or <code>std::vector::at()</code> (the latter throws an exception if the index is out of bounds).
Inserting at front	$\Theta(n)$	$\Theta(n)$	All n elements after the insertion point must be shifted right. <code>insert()</code> / <code>emplace()</code> with the begin iterator.
Inserting at back	$\Theta(1)$	$\Theta(n)$	No elements need shifting, but you may run out of capacity, forcing a reallocation and a copy of all n existing elements. <code>push_back()</code> / <code>emplace_back()</code>
Inserting at arbitrary index	$\Theta(n)$	$\Theta(n)$	On average an insertion lands in the middle, shifting about $n/2$ elements (still $\Theta(n)$). Worst case is at the front (all n shift). <code>insert()</code> / <code>emplace()</code> with an iterator.
Erasing at front	$\Theta(n)$	$\Theta(n)$	All n elements after the erasure point shift left so data starts at index 0 with no gaps. <code>erase()</code> with the begin iterator.
Erasing at back	$\Theta(1)$	$\Theta(1)$	Nothing shifts, and unlike insertion, erasing never triggers a reallocation on its own. <code>pop_back()</code>
Erasing at arbitrary index	$\Theta(n)$	$\Theta(n)$	About $n/2$ elements shift on average; worst case at the front shifts all n . <code>erase()</code> with an iterator or iterator range.
Checking if empty	$\Theta(1)$	$\Theta(1)$	Uses the vector's pointers to check whether the size is 0. <code>std::vector::empty()</code>